APUNTES DE DOCKER – DAW

APUNTES DE DOCKER – DAW **1**
1. Lo primero: ¿cómo se construye un comando? 2
2. Comandos más utilizados 2
1. IMÁGENES **3**
2. CONTENEDORES **3**
3. REDES **5**
4. VOLÚMENES **6**
5. CONSTRUIR IMÁGENES **7**

1. Lo primero: ¿cómo se construye un comando?

Ejemplo:

```
None
docker run -d --name servidor nginx:alpine
```

Parte	¿Qué significa?	¿Varía?
`docker run`	Crear y arrancar un contenedor	Fijo
`-d`	Ejecutarlo en segundo plano	Según el ejercicio
`--name`	Asignar un nombre	Fijo
`servidor`	Nombre del contenedor	Variable
`nginx:alpine`	Imagen utilizada	Variable

2. Comandos más utilizados

1. IMÁGENES

Descargar una imagen:

```
None
docker pull nginx:alpine
```

- Fijo: `docker pull`
- Variable: `nginx:alpine` (imagen y versión).

Ver imágenes descargadas:

```
None
docker images
```

- Comando fijo, no necesita cambiar nada.

Eliminar una imagen:

```
None
docker rmi nginx:alpine
```

- Fijo: `docker rmi`
- Variable: nombre de la imagen.

2. CONTENEDORES

Crear y arrancar un contenedor:

```
None
docker run -d --name servidor nginx:alpine
```

- Fijo: `docker run --name`
- Variable: `servidor` y `nginx:alpine`.
- `-d`: segundo plano.

Crear un contenedor interactivo:

```
None
docker run -it --name cliente alpine:latest sh
```

- `-it`: permite entrar y escribir comandos.
- Variables: nombre e imagen.
- `sh`: abre una terminal.

Ver contenedores funcionando:

```
None
docker ps
```

Ver todos, incluidos los detenidos:

```
None
docker ps -a
```

Parar un contenedor:

```
None
docker stop servidor
```

- Variable: nombre del contenedor.

Arrancar un contenedor parado:

```
None
docker start servidor
```

Eliminar un contenedor:

```
None
docker rm servidor
```

Eliminarlo aunque esté funcionando:

```
None
docker rm -f servidor
```

Entrar en un contenedor en ejecución:

```
None
docker exec -it servidor sh
```

- Variable: nombre del contenedor.
- `sh`: terminal.

3. REDES

Crear una red:

```
None
docker network create miSarea
```

- Fijo: `docker network create`
- Variable: nombre de la red.

Ver redes:

```
None
docker network ls
```

Crear un contenedor conectado a una red:

```
None
docker run -d --name servidor --network miSarea nginx:alpine
```

- Variables: nombre del contenedor, red e imagen.

Conectarse desde otro contenedor:

```
None
docker run -it --name cliente --network miSarea alpine:latest sh
```

Comprobar comunicación:

```
None
ping -c 3 servidor
```

- Fijo: `ping -c 3`
- Variable: nombre del contenedor de destino.

4. VOLÚMENES

Crear un volumen:

```
None
docker volume create Datos
```

- Variable: nombre del volumen.

Ver volúmenes:

```
None
docker volume ls
```

Crear un contenedor con volumen:

```
None
docker run -d --name mysql1 -e MYSQL_ROOT_PASSWORD=1234 -v
Datos:/var/lib/mysql mysql:8.0
```

- Variables: nombre, contraseña, volumen e imagen.
- `-v`: conecta el volumen con una carpeta del contenedor.
- `/var/lib/mysql`: carpeta donde MySQL guarda sus datos.

Eliminar un contenedor:

```
None
docker rm -f mysql1
```

Volver a crearlo usando el mismo volumen:

```
None
docker run -d --name mysql2 -e MYSQL_ROOT_PASSWORD=1234 -v
Datos:/var/lib/mysql mysql:8.0
```

IMPORTANTE: no elimines el volumen si quieres conservar los datos.

5. CONSTRUIR IMÁGENES

Construir una imagen desde un Dockerfile:

```
None
docker build -t text-utils:1.0 .
```

- Fijo: `docker build -t`
- Variable: `text-utils:1.0` (nombre y etiqueta).
- `.`: carpeta actual donde está el Dockerfile.

3. Instrucciones de un Dockerfile

Estas instrucciones no son comandos de PowerShell: se escriben dentro de un archivo llamado `Dockerfile`.

Instrucción	Para qué sirve	Qué cambia
`FROM`	Elegir imagen base	Imagen y versión
`WORKDIR`	Establecer carpeta de trabajo	Ruta
`COPY`	Copiar archivos	Archivo y destino
`RUN`	Ejecutar instalaciones o comandos al construir	Comando
`CMD`	Comando predeterminado al arrancar	Programa y argumentos
`ENTRYPOINT`	Definir el ejecutable principal	Programa y argumentos

Ejemplo completo:

```
None
FROM python:3.11-slim
WORKDIR /app
COPY procesar_texto.py /app/
RUN pip install nltk
ENTRYPOINT ["python", "procesar_texto.py"]
```

¿Qué cambiaría en otro ejercicio?

- Si pide Node.js: cambiarías la imagen `FROM`.
- Si pide `/web`: cambiarías `WORKDIR`.
- Si pide copiar `index.html`: cambiarías `COPY`.
- Si pide instalar Flask: cambiarías `RUN pip install nltk` por `RUN pip install flask`.
- Si pide ejecutar otro programa: cambiarías `ENTRYPOINT`.

4. Ejemplos de ejercicios típicos de examen

Ejercicio A. Crear un servidor web

Enunciado: crea un contenedor llamado `web` con Nginx funcionando en segundo plano.

```
None
docker run -d --name web nginx:alpine
```

Cambia: `web` (nombre) y `nginx:alpine` (imagen).

Ejercicio B. Publicar un puerto

Enunciado: crea Nginx y permite acceder desde el puerto 8080 de tu ordenador.

```
None
docker run -d --name web -p 8080:80 nginx:alpine
```

`8080` es el puerto de tu ordenador y `80` el del contenedor.

Ejercicio C. Crear una red y conectar dos contenedores

```
None
docker network create miSarea
docker run -d --name servidor --network miSarea nginx:alpine
docker run -it --name cliente --network miSarea alpine:latest sh
```

Dentro del cliente:

```
None
ping -c 3 servidor
```

Ejercicio D. Guardar datos con un volumen

```
None
docker volume create Datos
docker run -d --name mysql1 -e MYSQL_ROOT_PASSWORD=1234 -v
Datos:/var/lib/mysql mysql:8.0
```

Si eliminas `mysql1`, puedes crear otro contenedor utilizando `Datos` y recuperar la información.